

PROJECT DEVELOPMENT PHASE

ComicCraft – AI Comic Story Creator using Gemini Models

1. Development Overview

The Project Development Phase converts the planned ComicCraft design into working application components. Development covers the core AI generation functions, FastAPI backend, route handlers, frontend templates, image generation, comic layout construction, and PDF export.

2. Development Objectives

The implementation is organized to provide the following capabilities:

- Generate a structured five-panel comic outline from a user prompt.
- Expand the outline into narration and character dialogues.
- Generate comic-style illustrations for individual panels.
- Organize images and story content into a complete comic layout.
- Display the generated comic through a web interface.
- Export the complete comic as a downloadable PDF.

3. Development Architecture

ComicCraft is implemented using three connected layers: a frontend built with HTML, CSS, and Jinja2; a FastAPI backend responsible for routing and processing; and AI integration using Gemini models and Hugging Face Diffusers.

4. Development Flow

The development flow follows the sequence: user input → outline generation → story generation → image generation → layout construction → PDF export → web presentation.

PROJECT DEVELOPMENT PHASE – CORE AI DEVELOPMENT

1. Structured Comic Outline Development

The function `generate_outline()` in `gemini_flash.py` is developed to generate a structured five-panel comic outline based on the user's story prompt. Each panel contains a panel number, title, scene description, and an image generation prompt.

2. Story and Dialogue Development

The function `generate_story()` in `gemini_pro.py` expands the panel outlines into a full comic-style story. The implementation produces narration and character dialogues for the panels and returns the formatted story content.

3. Image Generation Development

The `generate_image()` function in `image_generator.py` uses the Stable Diffusion model to create a comic-style image from an image prompt. Generated files are sanitized for safe filenames and stored in the `static/panels` directory.

Development Component	Implementation
Outline Generation	Gemini Flash generates a structured 5-panel outline.
Story Generation	Gemini Pro creates narration and character dialogue.
Image Generation	Stable Diffusion creates panel illustrations.

4. Development Sequence

1. Generate the panel outline.
2. Use the outline to generate the detailed story.
3. Generate an illustration for each panel.
4. Store the generated images for later layout and export.

These modules keep text generation and image generation separate while allowing the backend to coordinate them as one comic-generation workflow.

PROJECT DEVELOPMENT PHASE – BACKEND AND ROUTES

1. FastAPI Backend Development

FastAPI is used to manage server-side operations and route handling. The backend receives user data, coordinates the AI generation functions, organizes the final comic layout, and supports PDF export.

2. User Input Processing

The HTML form collects the story prompt, main character name, setting, story tone, and art style. FastAPI captures these values using Form(...) parameters. For API-based generation, a Pydantic PromptRequest schema validates JSON input.

3. Route Development

Route	Development Function
/	Loads index.html and provides the comic input form.
/generate	Processes form input and runs the complete comic generation workflow.
/generate-comic/json	Accepts JSON input and returns comic layout data and PDF information.
/export-success	Displays confirmation after comic export.
/test-image	Tests image generation directly from a custom prompt.

4. Integrated Generation Workflow

The /generate and /generate-comic/json routes connect generate_outline(), generate_story(), generate_image(), build_comic_layout(), and save_pdf(). This integration allows the backend to transform user input into a complete comic and deliver the resulting data to the frontend.

5. Error Handling

The route handlers use FastAPI's HTTPException system and try-except blocks to handle unexpected failures, including API errors and JSON decoding errors.

PROJECT DEVELOPMENT PHASE – LAYOUT, FRONTEND AND PDF

1. Comic Layout Development

The `build_comic_layout()` function in `layout_builder.py` organizes the generated images and full comic story into a structured layout. It matches each image with its corresponding panel story and produces a list containing the panel number, image path, and text.

2. PDF Export Development

The `save_pdf()` function in `exporters.py` compiles the completed comic into a multi-page PDF using the FPDF library. Each panel's image and narration are placed into the exported document.

3. Frontend Development

The frontend is developed with HTML, CSS, and Jinja2 templates. The three main templates support comic creation, preview, and export confirmation.

Template	Development Role
index.html	Collects story prompt, character name, setting, tone, and art style.
comic_preview.html	Displays the generated comic panel-by-panel.
export_success.html	Confirms successful PDF export and provides navigation.

4. Dynamic Jinja2 Rendering

Jinja2 templates dynamically receive backend-generated content. The comic preview uses a loop over the layout list to display the panel number, title, image, scene description, captions, and narration.

5. User Interface Development

The interface uses organized form fields, consistent button styling, padding, shadows, rounded corners, and a responsive layout approach to create a clean comic-generation experience.

PROJECT DEVELOPMENT PHASE – INTEGRATION AND VERIFICATION

1. Full System Integration

The completed development connects frontend input, FastAPI processing, AI generation, image storage, layout construction, and PDF export into one workflow. Generated content is returned to the frontend for preview and can then be exported.

2. Development Verification

The application is designed to be run locally using Uvicorn. The development verification process checks the homepage, comic-generation route, JSON API, image-generation test route, preview page, and PDF export flow.

3. Local Execution

The project uses the FastAPI application entry point with Uvicorn:

```
uvicorn app.main:app --reload
```

The local application is accessed through 127.0.0.1:8000, while the FastAPI interactive API documentation is available through the /docs endpoint.

4. Development Completion Checklist

- ✓ Core outline generation function developed.
- ✓ Story and dialogue generation function developed.
- ✓ Stable Diffusion image generation function developed.
- ✓ Comic layout construction developed.
- ✓ PDF export function developed.
- ✓ FastAPI routes integrated with the AI workflow.
- ✓ Jinja2 templates connected to backend-generated data.
- ✓ Local deployment and verification prepared.

5. Development Outcome

The Project Development Phase results in an integrated ComicCraft application that can accept a creative prompt, generate a panel-based story, create corresponding illustrations, display the comic through a web interface, and produce a downloadable PDF.

The modular implementation also provides a foundation for future enhancements identified in the project, such as user profiles, comic libraries, and multi-page story arcs.

Conclusion

The development phase transforms the ComicCraft design and plan into connected software components, covering AI generation, backend routing, frontend rendering, layout management, and PDF export.